Subject 0: Prompt Engineering Foundations for Beginners

Welcome to Your AI Learning Journey!

This document is your starting point for learning prompt engineering. **No prior experience required!** By the end of this 4-week foundation course, you'll be ready to advance to production-level prompt engineering.

Who This Course Is For

✅ **Perfect if you:**
- Are curious about AI and want hands-on experience
- Have little to no programming experience
- Want to understand how to work with ChatGPT, Claude, or similar AI tools
- Are exploring AI as a potential career path
- Want to use AI effectively in your current job

❌ **Skip ahead if you:**
- Already build applications with LLM APIs
- Are comfortable with Python, JSON, and APIs
- Have experience with version control (git)
- → **If this is you, start with Subject 1: Prompt Engineering as an Engineering Discipline**

What You'll Learn

By the end of this course, you will:
- Understand what Large Language Models (LLMs) are and how they work
- Write effective prompts for different use cases
- Iterate and improve prompts systematically
- Use AI APIs programmatically (basic Python)
- Recognize when AI is and isn't the right tool
- Be ready for production-level prompt engineering training

Time Commitment

- **4 weeks total**
- **5-8 hours per week**
- Mix of reading, hands-on practice, and experimentation

What You Need

Required:
- Computer with internet access
- Web browser (Chrome, Firefox, or Safari)
- Curiosity and willingness to experiment

Free tools we'll use:
- ChatGPT (free tier) or similar AI chatbot
- Google Colab (free, no installation needed)
- Text editor (Notepad, TextEdit, or similar)

Optional (we'll guide you through setup):
- Python (free programming language)
- API access to OpenAI or Anthropic ($5-10 for the course)

Week 1: Understanding AI and Large Language Models

Learning Objectives

By the end of this week, you will:
- Explain what AI and LLMs are in simple terms
- Identify real-world applications of LLMs
- Use ChatGPT or similar tools for basic tasks
- Understand the limitations and risks of AI systems

Day 1: What is Artificial Intelligence?

Core Concepts

Artificial Intelligence (AI) is technology that enables computers to perform tasks that typically require human intelligence.

Examples you use every day:
- **Autocomplete** on your phone (predicts next word)
- **Voice assistants** like Siri or Alexa
- **Recommendation systems** on Netflix or Spotify
- **Spam filters** in your email

What makes something "intelligent"?
- Learning from examples
- Recognizing patterns
- Making predictions
- Adapting to new situations

Types of AI

1. **Narrow AI** (What exists today)
   - Designed for specific tasks
   - Examples: spam detection, image recognition, playing chess
   - Cannot transfer learning to unrelated tasks

2. **General AI** (Doesn't exist yet)
   - Hypothetical AI that can do any intellectual task a human can
   - Science fiction territory for now

Key insight: All AI systems today are narrow AI, even ChatGPT. They're incredibly good at specific things but have clear limitations.

Hands-On Activity: Observing AI in Your Life

Task: Document 5 places you encountered AI today.

Example:
1. Gmail sorted an email into spam (spam filter)
2. YouTube recommended a video (recommendation algorithm)
3. My phone suggested "I'm on my way" when texting (autocomplete)
4. Instagram showed me targeted ads (ad targeting)
5. My banking app flagged an unusual transaction (fraud detection)

Reflection question: What patterns do you notice? (Most AI helps with sorting, predicting, or recommending)

Day 2: What Are Large Language Models?

Core Concepts

Large Language Model (LLM) = AI system trained on massive amounts of text to understand and generate human language.

Popular LLMs:
- **ChatGPT** (by OpenAI) - Most well-known
- **Claude** (by Anthropic) - Known for safety and longer conversations
- **Gemini** (by Google) - Integrated with Google services
- **Llama** (by Meta) - Open source

How do they work? (Simple explanation)

1. **Training Phase:**
   - Read billions of web pages, books, articles
   - Learn patterns: "After 'The cat sat on the', 'mat' is likely"
   - Learn grammar, facts, writing styles, reasoning patterns
2. **Using Phase:**
   - You give it a prompt (instruction or question)
   - It predicts the most likely continuation
   - Generates text word-by-word based on patterns learned

Key insight: LLMs don't "know" things like humans do. They recognize patterns from training data. They're like incredibly sophisticated autocomplete.

What LLMs Can Do Well

✅ **Strong at:**
- Writing (emails, articles, creative content)
- Summarizing long texts
- Translation between languages
- Explaining concepts in different ways
- Brainstorming ideas
- Basic coding help
- Answering factual questions (usually)

What LLMs Can't Do (Or Do Poorly)

❌ **Weak at:**
- Math (they can get arithmetic wrong!)
- Real-time information (trained on old data)
- Personal information about you (unless you tell them)
- Tasks requiring up-to-the-second accuracy
- Understanding images (unless specifically designed for it)
- Truly "understanding" like humans do

Hands-On Activity: Your First LLM Interaction

Tool: ChatGPT (https://chat.openai.com) - Free tier is fine

Task 1: Simple request

```
Prompt: Explain quantum physics to a 5-year-old.
```

Observe:
- How quickly does it respond?
- Is the explanation actually simple?
- Try clicking "Regenerate" - does the answer change?

Task 2: Test its limitations

```
Prompt: What is the current weather in [your city]?
```

Observe:
- It probably says it can't access real-time data
- This is a key limitation to remember

Task 3: Creative task

```
Prompt: Write a haiku about coffee.
```

Observe:
- Does it follow the 5-7-5 syllable pattern?
- Is it creative or generic?

Reflection: Write down 3 observations about how the LLM responds.

Day 3: How LLMs Are Trained (Non-Technical)

The Training Process (Simplified)

Step 1: Gather Text Data
- Billions of web pages, books, articles, code
- Think: "Reading the entire internet"
- Data up to a cutoff date (e.g., ChatGPT-4 trained on data until April 2023)

Step 2: Pattern Learning
- The AI finds patterns in language
- Example: "The capital of France is ____" → "Paris" appears often
- Learns grammar, facts, styles, reasoning patterns

Step 3: Fine-Tuning
- Human reviewers rate AI outputs
- AI learns what humans prefer
- This is why ChatGPT tries to be helpful and polite

Important concepts:

Training data cutoff:
- LLMs don't know about events after their training
- Example: A model trained in 2023 doesn't know about 2024 events

Bias in training data:
- If training data contains biases, the AI learns them
- Example: Biased hiring descriptions → AI generates biased content
- Ongoing challenge in AI development

Emergent abilities:
- LLMs can do things they weren't explicitly trained for
- Example: Trained on English and Spanish texts separately, but can translate between them
- Why this happens is still being researched!

Analogy: LLMs as "Lossy Compression" of the Internet

Imagine:
- You read 10,000 books
- You can't remember every word, but you understand patterns
- Someone asks a question, you generate an answer based on patterns

LLMs work similarly:
- "Read" the internet during training

- Compress knowledge into patterns (the model's "weights")
- Generate answers based on those patterns

Key limitation: They might confidently generate incorrect information (called "hallucinations") because they're predicting likely text, not retrieving facts.

Hands-On Activity: Detecting Training Cutoffs

Task: Test ChatGPT's knowledge cutoff

Try these prompts:

```
1. Who won the 2020 US Presidential election?
2. Who won the 2024 US Presidential election?
3. What is the latest iPhone model?
```

Observe:
- Can it answer some but not others?
- Does it tell you it has a knowledge cutoff?
- What date is its cutoff?

Reflection: Why does this matter for real-world applications?

Day 4: Key Terminology for Working with LLMs

Essential Terms (You'll Use These Daily)

Prompt:
- The text you give to an LLM as input
- Can be a question, instruction, or partial text
- Example: "Summarize this article in 3 sentences"

Completion / Response / Output:
- The text the LLM generates in response to your prompt
- Example: The summary it writes

Token:
- A chunk of text (roughly ¾ of a word)
- LLMs process text as tokens, not words
- "Hello world" = ~2 tokens
- "Artificial intelligence" = ~3 tokens
- **Why it matters:** API pricing is per token, not per word

Context / Context Window:
- How much text the LLM can "remember" at once
- Includes your prompt + previous conversation + its response
- Example: GPT-3.5 can handle ~4,000 tokens (~3,000 words)
- Think of it like short-term memory

Temperature:
- Controls randomness/creativity of outputs
- Scale: 0.0 to 2.0
- Low (0.0): Deterministic, consistent, focused
- High (1.5+): Creative, varied, unpredictable
- Default: Usually around 0.7

Hallucination:
- When an LLM confidently generates false information
- Happens because it predicts likely text, not truth
- Example: Inventing fake sources or statistics
- **Critical to watch for in production use**

Fine-tuning:
- Training an LLM further on specific data
- Makes it better at specific tasks
- More advanced topic (we'll cover basics in Week 4)

API Terms (We'll Use These in Weeks 3-4)

API (Application Programming Interface):
- A way for programs to talk to each other
- Think: A menu at a restaurant (you order from the menu, kitchen makes it)
- LLM API: You send a prompt, get back a completion

Endpoint:
- The specific URL you send requests to
- Example: `https://api.openai.com/v1/chat/completions`

API Key:
- Your secret password for using an API
- Proves you have permission and tracks your usage
- **Never share publicly!**

Rate Limit:
- Maximum number of requests you can make in a time period
- Example: 3 requests per minute on free tier
- Prevents abuse and manages server load

Hands-On Activity: Understanding Tokens

Tool: OpenAI Tokenizer (https://platform.openai.com/tokenizer)

Task: Test how text is tokenized

Paste these texts and observe token counts:
1. "Hello, world!"
2. "artificial intelligence"
3. A full paragraph from a news article
4. Code snippet (try Python code)

Observations to note:
- How many tokens is an average sentence?
- Do spaces count as tokens?
- Are common words fewer tokens than rare words?

Why this matters:
- If API costs $0.002 per 1,000 tokens
- And your prompt + response = 500 tokens
- Each API call costs $0.001

Day 5: Safety, Ethics, and Limitations

What Can Go Wrong?

1. Hallucinations (Confidently Wrong Information)

Example:

```
Prompt: Who won the Nobel Prize in Physics in 2023?
Bad response: Dr. Jane Smith won for her work on quantum teleportation.
```

(This is fabricated if Jane Smith doesn't exist or didn't win)

How to mitigate:
- Always verify critical information
- Use LLMs for drafting, not as authoritative sources
- Cross-reference with reliable sources

2. Bias and Fairness Issues

Example scenarios:
- Job description generator uses biased language ("rockstar developer" → discourages some groups)
- Resume screener inherits biases from training data
- Content moderation inconsistent across languages/cultures

How to mitigate:
- Review AI outputs for bias
- Test with diverse inputs
- Have human oversight for important decisions

3. Privacy Concerns

Key rules:
- Don't input confidential information into public LLMs
- Assume your prompts might be used for training (read terms of service)
- Use enterprise/private APIs for sensitive work

4. Overreliance on AI

Dangers:
- Accepting incorrect outputs without verification
- Losing critical thinking skills
- Automation bias (trusting AI over your judgment)

Best practice: Use AI as a tool to augment your abilities, not replace your thinking.

Responsible AI Use Guidelines

DO:
- ✅ Verify factual claims, especially for important decisions
- ✅ Disclose when content is AI-generated (when relevant)
- ✅ Review outputs for quality, bias, and accuracy
- ✅ Use AI to speed up work, not skip critical thinking
- ✅ Keep learning about AI limitations and capabilities

DON'T:
- ❌ Submit AI-generated academic work as your own
- ❌ Use AI for medical, legal, or financial advice without expert review
- ❌ Input personal/confidential information
- ❌ Assume AI outputs are factually correct
- ❌ Use AI to manipulate, deceive, or harm others

Hands-On Activity: Testing for Hallucinations

Task: Make ChatGPT hallucinate, then verify

Try these prompts:

```
1. List the 5 books written by [make up a fake author name]
2. What are the side effects of [make up a fake medication]?
3. Summarize the plot of the movie "[make up a fake movie title]"
```

Observe:
- Does it confidently provide fake information?
- Does it admit when it doesn't know?
- How convincing are the hallucinations?

Then verify:
- Google the fake names you made up
- Did the LLM invent plausible-sounding but false details?

Key lesson: Always verify AI outputs, especially factual claims.

Day 6-7: Week 1 Project

Project: AI Capability Assessment

Goal: Systematically test what an LLM can and cannot do.

Instructions:

1. **Choose 10 tasks across different categories:**
   - 2 factual questions (one recent, one historical)
   - 2 creative tasks (writing, brainstorming)
   - 2 analytical tasks (summarizing, explaining)
   - 2 tasks requiring current information
   - 2 tasks testing limitations (math, reasoning)

2. **For each task:**
   - Write your prompt
   - Record the LLM's response
   - Rate quality: ★☆☆ (Poor), ★★☆ (Good), ★★★ (Excellent)
   - Note any hallucinations or errors
   - Identify one way to improve the prompt

3. **Deliverable:** A simple table (can be in a document or spreadsheet)

Example entry:

Task	Prompt	Response Quality	Notes	Improvement Idea
Factual	"What is photosynthesis?"	★★★	Accurate, clear explanation	Could ask for simpler language
Current info	"What's the weather today?"	★☆☆	Can't access real-time data	Understand this is a limitation, use weather API instead
Creative	"Write a haiku about rain"	★★☆	Creative but syllables wrong	Specify "exactly 5-7-5 syllables"

Reflection questions:
1. Which tasks did the LLM excel at?
2. Which tasks exposed clear limitations?
3. Did you notice any patterns in what works vs. doesn't work?
4. How would you explain LLMs to a friend after this experiment?

Week 2: Writing Effective Prompts

Learning Objectives

By the end of this week, you will:
- Write clear, specific prompts
- Use examples to improve LLM outputs
- Iterate on prompts systematically
- Apply prompting techniques for different use cases
- Understand when to break complex tasks into steps

Day 8: The Anatomy of a Good Prompt

What Makes a Prompt Effective?

Poor prompt:

```
Tell me about dogs.
```

Why it's poor:
- Too vague (What aspect of dogs?)
- No context (Why do you need this info?)
- No constraints (How long? What style?)

Good prompt:

```
Write a 150-word informative paragraph about golden retrievers suitable for a family
considering adopting one. Focus on temperament, care requirements, and compatibility
with children. Use a friendly but factual tone.
```

Why it's good:
- Specific topic (golden retrievers, not all dogs)
- Clear purpose (helping family decide)
- Defined constraints (150 words, specific topics)
- Specified tone (friendly but factual)

The 5 Elements of Effective Prompts

1. ROLE (Who should the AI be?)

```
"You are a patient elementary school teacher explaining science concepts."
"You are a professional business consultant."
```

2. TASK (What should it do?)

```
"Summarize this article"
"Generate 5 creative ideas"
"Proofread this email"
```

3. CONTEXT (Background information)

```
"I'm writing to a potential client who expressed interest last month."
"This is for a homework assignment on climate change."
```

4. FORMAT (How should output be structured?)

```
"Provide output as a bulleted list"
"Format as a professional email"
"Create a numbered step-by-step guide"
```

5. CONSTRAINTS (Limits and requirements)

```
"Keep under 200 words"
"Use language suitable for teenagers"
"Avoid technical jargon"
```

Hands-On Activity: Before and After

Task: Rewrite these poor prompts using the 5 elements

Poor prompt 1: "How do I learn Python?"

Your improved prompt:

```
[Role]: You are an experienced programming instructor
[Task]: Create a learning plan for Python
[Context]: I'm a complete beginner with no programming experience, want to build web app
[Format]: Provide as a 4-week structured plan with weekly goals
[Constraints]: Each week should require 5-8 hours; include free resources only
```

Poor prompt 2: "Make this better: [paste email]"

Your improved prompt:

```
[Role]: You are a professional communications editor
[Task]: Improve this email for clarity and professionalism
[Context]: This email is to my manager requesting time off
[Format]: Provide the revised email, then bullet points explaining changes
[Constraints]: Keep the same core message; maintain polite but confident tone
```

Practice: Take 3 prompts you used in Week 1 and rewrite them using this framework.

Day 9: Few-Shot Prompting (Learning from Examples)

What is Few-Shot Prompting?

Concept: Show the LLM examples of what you want, then ask it to do the same thing.

Why it works: LLMs are pattern matchers. Examples clarify your expectations better than instructions alone.

Zero-Shot vs. Few-Shot

Zero-shot (no examples):

```
Classify the sentiment of this review as positive, negative, or neutral:
"The food was okay but service was slow."
```

One-shot (1 example):

```
Classify sentiment as positive, negative, or neutral.

Example:
Review: "Best pizza I've ever had!"
Sentiment: Positive

Now classify:
Review: "The food was okay but service was slow."
Sentiment:
```

Few-shot (multiple examples):

```
Classify sentiment as positive, negative, or neutral.

Review: "Best pizza I've ever had!"
Sentiment: Positive

Review: "Terrible experience, never going back."
Sentiment: Negative

Review: "It was fine, nothing special."
Sentiment: Neutral
```

```
Review: "The food was okay but service was slow."
Sentiment:
```

Result: Few-shot typically performs better, especially for nuanced tasks.

When to Use Few-Shot Prompting

Use few-shot when:
- ✅ You need consistent formatting
- ✅ The task is ambiguous or subjective
- ✅ You have specific style requirements
- ✅ Zero-shot results are inconsistent

Skip few-shot when:
- ❌ The task is very straightforward ("Translate to Spanish")
- ❌ You don't have good examples
- ❌ Context window is limited (examples take up space)

Hands-On Activity: Few-Shot Experiments

Task: Test zero-shot vs. few-shot for email classification

Scenario: You receive customer emails and need to route them to correct departments.

Your emails to classify:
1. "I can't log into my account, getting error code 403"
2. "How much does the premium plan cost?"
3. "I love your product! Just wanted to say thanks."
4. "When will the new feature be released?"

Step 1: Try zero-shot

```
Classify these customer emails as: Technical Support, Sales, Feedback, or Product Inquir

[paste emails above]
```

Record accuracy.

Step 2: Try few-shot

```
Classify customer emails as: Technical Support, Sales, Feedback, or Product Inquiry

Examples:
Email: "My payment didn't go through, card was charged twice"
```

```
Category: Technical Support

Email: "Do you offer educational discounts?"
Category: Sales

Email: "Your app saved me hours of work!"
Category: Feedback

Email: "Does the pro version include API access?"
Category: Product Inquiry

Now classify:
[paste emails above]
```

Compare:
- Did few-shot improve accuracy?
- Did it change formatting consistency?
- Which approach felt more reliable?

Day 10: Chain-of-Thought Prompting

What is Chain-of-Thought?

Concept: Ask the LLM to show its reasoning step-by-step before giving a final answer.

Why it works:
- Forces systematic thinking
- Reduces logical errors
- Makes outputs easier to verify
- Improves performance on complex tasks

Without Chain-of-Thought

```
Prompt: If a train leaves Station A at 2pm traveling 60mph, and another leaves
Station B (300 miles away) at 3pm traveling 80mph toward each other, when do they meet?

Response: They meet at 4:17pm.
```

Problem: Answer might be wrong, but you can't tell why.

With Chain-of-Thought

```
Prompt: If a train leaves Station A at 2pm traveling 60mph, and another leaves
Station B (300 miles away) at 3pm traveling 80mph toward each other, when do they meet?

Think through this step-by-step before answering.

Response:
Let me work through this step-by-step:

1. First train leaves at 2pm, second at 3pm (1 hour difference)
2. In that first hour, Train A travels: 60 mph × 1 hour = 60 miles
3. Remaining distance between trains: 300 - 60 = 240 miles
4. Combined speed when both moving: 60 + 80 = 140 mph
5. Time to close remaining gap: 240 miles ÷ 140 mph = 1.71 hours = 1 hour 43 minutes
6. Add to 3pm: 3:00 + 1:43 = 4:43pm

Answer: They meet at 4:43pm.
```

Benefits: You can verify each step, catch errors in reasoning.

How to Trigger Chain-of-Thought

Explicit instruction:
- "Think step-by-step"
- "Let's approach this systematically"
- "Show your reasoning before answering"

Example-based (few-shot with reasoning):

```
Question: What is 15% of 80?
Reasoning:
- 10% of 80 = 8
- 5% of 80 = 4
- 15% = 10% + 5% = 8 + 4 = 12
Answer: 12

Question: What is 22% of 150?
[LLM follows the pattern and shows reasoning]
```

Hands-On Activity: Debugging with Chain-of-Thought

Task: Find the error in LLM reasoning

Prompt:

```
A store has a 25% off sale. After the sale, they raise prices by 25%.
Are items back to original price?

Show your step-by-step reasoning.
```

Observe the response:
- Does it reason correctly?
- Common mistake: Assuming 25% off then 25% up = original price (mathematically incorrect!)
- Correct reasoning: 25% off of 100 = 75, then 25% increase on 75 = 93.75 (not 100)

Try this with 3 different LLMs (ChatGPT, Claude, Gemini) - do they all get it right?

Day 11: Prompt Iteration and Refinement

The Prompt Engineering Cycle

Process:
1. Write initial prompt
2. Test with real inputs
3. Identify failures
4. Refine prompt
5. Repeat until satisfactory

Key insight: Your first prompt will rarely be perfect. Treat it as a draft.

Common Failure Modes and Fixes

Failure Mode 1: Output too vague

Problem prompt:

```
Summarize this article.
```

Fixed prompt:

```
Summarize this article in exactly 3 bullet points. Each bullet should:
- Be one complete sentence
```

```
- Highlight a key finding
- Be understandable without reading the article
```

Failure Mode 2: Wrong tone

Problem prompt:

```
Write a response to this customer complaint.
```

Fixed prompt:

```
Write a response to this customer complaint. Tone should be:
- Empathetic and apologetic
- Professional but warm
- Solution-focused
Avoid: defensive language, corporate jargon, generic responses
```

Failure Mode 3: Inconsistent formatting

Problem prompt:

```
Extract key information from these resumes.
```

Fixed prompt:

```
Extract key information from these resumes in this exact format:

Name: [Full name]
Email: [Email address]
Years of experience: [Number]
Top 3 skills: [Skill 1], [Skill 2], [Skill 3]

If information is missing, write "Not provided"
```

Hands-On Activity: Systematic Iteration

Task: Improve a prompt through 3 iterations

Scenario: You want to generate social media posts from blog articles.

Iteration 1 (Baseline):

```
Turn this blog post into a tweet.
```

Test with 3 different blog posts. Record what goes wrong:
- Too long? Too short?
- Wrong tone?
- Misses key points?

Iteration 2 (Add constraints):

```
Turn this blog post into a tweet:
- Maximum 240 characters (leaving room for link)
- Include one relevant emoji
- Write in an engaging, conversational tone
- Highlight the main benefit or insight
```

Test again. What improved? What still needs work?

Iteration 3 (Add examples and format):

```
Turn this blog post into a tweet following this structure:

[Hook question or statement] [Key insight] [Emoji] [Call-to-action]

Example:
Blog topic: Time management tips
Tweet: "Struggling with endless to-do lists? ⏰ The 2-minute rule: if it takes <2 min, d

Now create a tweet from this blog post:
[paste blog]
```

Reflection:
- How much did each iteration improve results?
- Which changes had the biggest impact?
- Is the prompt getting too long/complex?

Day 12: Advanced Prompting Techniques

Technique 1: Role + Persona

Basic role:

```
You are a teacher.
```

Enhanced with persona:

```
You are an enthusiastic high school biology teacher known for making complex topics
accessible through real-world examples and humor. You adapt explanations based on
student confusion and always encourage questions.
```

Impact: More consistent personality and style in outputs.

Technique 2: Constrain with Negative Examples

Tell it what NOT to do:

```
Write a professional email requesting a deadline extension.

Do NOT:
- Make excuses or sound whiny
- Be overly apologetic
- Use phrases like "I know you're busy but..."
- Be vague about the new deadline needed

DO:
- Be direct and confident
- Acknowledge the situation professionally
- Propose a specific new deadline
- Offer a solution or compromise
```

Technique 3: Format Enforcement

For consistent parsing:

```
Extract information in this EXACT format:

Company: [company name]
Position: [job title]
Location: [city, state]
Salary: [range or "Not listed"]

Example output:
Company: TechCorp
Position: Senior Engineer
```

```
Location: Austin, TX
Salary: $120k-150k

Now extract from this job posting:
[paste job posting]
```

Technique 4: Conditional Logic

Guide the LLM through decision trees:

```
Analyze this customer review and:

IF sentiment is positive:
  - Thank the customer
  - Highlight the specific praised feature
  - Encourage them to share their experience

IF sentiment is negative:
  - Apologize specifically for the issue mentioned
  - Offer a concrete solution
  - Provide contact info for further assistance

IF sentiment is neutral:
  - Acknowledge their feedback
  - Ask specific questions to understand their needs better

Review: [paste review]
```

Hands-On Activity: Technique Comparison

Task: Compare different techniques for the same task

Goal: Generate product descriptions from bullet points

Test each approach:

Approach A (Basic):

```
Turn these product features into a description:
- Waterproof
- 12-hour battery
- Bluetooth 5.0
```

Approach B (Role + persona):

```
You are an experienced e-commerce copywriter known for benefit-focused, conversational
product descriptions.

Turn these features into a description:
[same features]
```

Approach C (Few-shot):

```
Turn product features into descriptions.

Example:
Features:
- Noise-cancelling
- Wireless
- 20-hour battery

Description:
"Immerse yourself in pure sound with active noise-cancelling technology. Cut the cord
with seamless wireless connectivity, and keep the music going for 20 full hours on a
single charge."

Now create a description for:
[same features]
```

Evaluate:
- Which approach produced the best description?
- Which was most consistent across multiple products?
- What's the tradeoff between simplicity and control?

Day 13-14: Week 2 Project

Project: Build a Prompt Library

Goal: Create a reusable collection of prompts for common tasks

Instructions:

1. **Choose 5 common use cases** from your life/work:
   - Examples: Email writing, summarizing articles, brainstorming ideas, proofreading, explaining concepts

2. **For each use case, create:**
   - **Base prompt template** (with placeholders for variables)
   - **3 test examples** (real inputs)
   - **Quality criteria** (what makes a good output?)
   - **Iteration notes** (what versions did you try?)

3. **Document in a simple format:**

Example entry:

Use Case: Professional Email Writing

Base Prompt Template:

```
You are a professional communications specialist. Write a [TONE] email for the
following situation:

Recipient: [RECIPIENT_ROLE]
Purpose: [PURPOSE]
Key points to include: [KEY_POINTS]
Desired outcome: [OUTCOME]

Email should be:
- Concise (under 150 words)
- Professional but personable
- Clear call-to-action

Context: [ADDITIONAL_CONTEXT]
```

Test Example 1:
- Recipient: Manager
- Purpose: Request flexible work arrangement
- Key points: Productivity data, family situation, proposed schedule
- Outcome: Approval for 2 days/week remote work

Output Quality: ★★★☆☆
- Strength: Professional tone, clear structure
- Weakness: Too formal, could be more personable
- Next iteration: Add persona detail about conversational style

Iteration Notes:
- V1: Basic "write an email" → too generic
- V2: Added structure and constraints → better formatting

- V3: Added tone guidance and word limit → current version
- Future: Test few-shot examples for consistency

Deliverable Format Options:
- Simple text document with sections for each use case
- Spreadsheet with columns: Use Case, Prompt, Variables, Examples, Quality Notes
- Note-taking app (Notion, Evernote, etc.)

Bonus Challenge:
- Test your prompts with 2 different LLMs (ChatGPT and Claude)
- Document which prompts work consistently across models
- Note which require model-specific adjustments

Week 3: Introduction to Programming with LLMs

Learning Objectives

By the end of this week, you will:
- Understand basic programming concepts (no prior experience needed)
- Write simple Python scripts to call LLM APIs
- Automate prompt testing with code
- Save and process LLM outputs programmatically

Note: This week moves from no-code to low-code. Take your time and don't worry if concepts feel challenging at first.

Day 15: Programming Basics (Python Crash Course)

Why Learn Programming for Prompt Engineering?

What you can do with code that you can't do manually:
- Test 100 prompts automatically instead of manually
- Process large datasets (summarize 1,000 articles)
- Build applications that use LLMs
- Track costs and performance systematically
- Create tools others can use

Key insight: You don't need to become a software engineer. You need just enough Python to automate LLM interactions.

Core Programming Concepts

1. Variables (Storing Information)

```
# A variable is like a labeled box that holds information
name = "Alice"
age = 25
is_student = True

# You can use variables later
print(name)  # Shows: Alice
```

Think of it as: Giving a nickname to information so you can refer to it easily.

2. Data Types

```
# Text (called "strings")
greeting = "Hello, world!"

# Numbers (integers)
count = 42

# Numbers with decimals (floats)
price = 19.99

# True/False values (booleans)
is_complete = True

# Lists (multiple items)
fruits = ["apple", "banana", "orange"]
```

3. Functions (Reusable Actions)

```
# Define a function
def greet(name):
    message = "Hello, " + name + "!"
    return message

# Use the function
```

```
result = greet("Alice")
print(result)  # Shows: Hello, Alice!
```

Think of it as: A recipe you can follow whenever needed, just changing the ingredients.

4. Conditionals (Making Decisions)

```
temperature = 75

if temperature > 80:
    print("It's hot!")
elif temperature > 60:
    print("It's pleasant")
else:
    print("It's cold!")
```

5. Loops (Repeating Actions)

```
# Repeat for each item in a list
fruits = ["apple", "banana", "orange"]

for fruit in fruits:
    print("I like " + fruit)

# Output:
# I like apple
# I like banana
# I like orange
```

Hands-On Activity: Your First Python Program

Tool: Google Colab (https://colab.research.google.com/)
- Free, runs in browser, no installation needed
- Click "New Notebook"

Task 1: Hello World

```
# Click in the cell below and press Shift+Enter to run

print("Hello, World!")
print("I'm learning prompt engineering!")
```

Task 2: Variables and Math

```
# Calculate cost of API calls

calls_per_day = 100
cost_per_call = 0.002

daily_cost = calls_per_day * cost_per_call
monthly_cost = daily_cost * 30

print("Daily cost: $" + str(daily_cost))
print("Monthly cost: $" + str(monthly_cost))
```

Task 3: Working with Lists

```
# Prompt versions to test
prompts = [
    "Summarize this article briefly",
    "Write a 3-sentence summary of this article",
    "Provide a concise summary in 50 words or less"
]

# Print each one
for i, prompt in enumerate(prompts):
    print(f"Version {i+1}: {prompt}")
```

Task 4: Simple Function

```
# Function to format a prompt
def create_summary_prompt(text, word_limit):
    prompt = f"Summarize the following in {word_limit} words or less:\n\n{text}"
    return prompt

# Test it
article = "AI is transforming industries..."
result = create_summary_prompt(article, 50)
print(result)
```

Reflection:

- Which concept felt most confusing? (This is normal!)
- Can you think of a task you'd like to automate?

Day 16: APIs and Making Your First LLM Call

What is an API?

Analogy: Restaurant
- **You (the customer):** Your code
- **Menu:** API documentation
- **Waiter:** The API
- **Kitchen:** The LLM
- **Food:** The response

You → Order (request) → Waiter → Kitchen → Food (response) → You

In LLM terms:
- You send a prompt (request)
- API delivers to LLM
- LLM generates response
- API returns response to you

Getting API Access

Option 1: OpenAI (ChatGPT)

1. Go to https://platform.openai.com/signup
2. Create account
3. Navigate to API keys section
4. Create new secret key
5. Copy it (you'll only see it once!)
6. Add $5-10 credit to your account

Option 2: Anthropic (Claude) - Similar process
- https://console.anthropic.com/

Security rule: NEVER share your API key or commit it to public code!

Your First API Call

In Google Colab:

```
# Step 1: Install the library
!pip install openai

# Step 2: Import it
```

```
import openai

# Step 3: Set your API key (replace with your actual key)
openai.api_key = "sk-your-key-here"  # KEEP THIS SECRET!

# Step 4: Make a request
response = openai.ChatCompletion.create(
    model="gpt-3.5-turbo",
    messages=[
        {"role": "user", "content": "Explain APIs in one sentence"}
    ]
)

# Step 5: Get the answer
answer = response['choices'][0]['message']['content']
print(answer)
```

What just happened?

1. You sent a message to OpenAI's servers
2. Their LLM processed it
3. Response came back as JSON (structured data)
4. You extracted the text content

Understanding the API Response

The full response object looks like:

```
{
  "id": "chatcmpl-123",
  "object": "chat.completion",
  "created": 1677652288,
  "model": "gpt-3.5-turbo",
  "choices": [{
    "index": 0,
    "message": {
      "role": "assistant",
      "content": "APIs are interfaces that let programs communicate."
    },
    "finish_reason": "stop"
  }],
  "usage": {
    "prompt_tokens": 13,
    "completion_tokens": 7,
    "total_tokens": 20
```

```
  }
}
```

Key parts:

- `choices[0].message.content` = The actual text response
- `usage.total_tokens` = How much this cost (in tokens)
- `model` = Which LLM was used

Hands-On Activity: Experimenting with Parameters

Task: See how parameters change outputs

```
import openai

openai.api_key = "your-key-here"

prompt = "Write a creative story opening about a robot."

# Test 1: Low temperature (more focused)
response1 = openai.ChatCompletion.create(
    model="gpt-3.5-turbo",
    messages=[{"role": "user", "content": prompt}],
    temperature=0.2
)
print("Low temperature:", response1['choices'][0]['message']['content'])

# Test 2: High temperature (more creative)
response2 = openai.ChatCompletion.create(
    model="gpt-3.5-turbo",
    messages=[{"role": "user", "content": prompt}],
    temperature=1.5
)
print("\nHigh temperature:", response2['choices'][0]['message']['content'])

# Test 3: With max tokens (shorter response)
response3 = openai.ChatCompletion.create(
    model="gpt-3.5-turbo",
    messages=[{"role": "user", "content": prompt}],
    max_tokens=50
)
print("\nWith token limit:", response3['choices'][0]['message']['content'])
```

Parameters to experiment with:

- `temperature` : 0.0-2.0 (creativity level)

- `max_tokens`: Limit response length
- `top_p`: Alternative to temperature (nucleus sampling)
- `n`: Generate multiple responses

Day 17: Automating Prompt Testing

Why Automate Testing?

Manual testing:
- You: Type prompt into ChatGPT
- You: Copy response
- You: Paste into document
- You: Repeat 20 times
- **Time: 30-60 minutes**

Automated testing:
- Code: Runs 20 prompts in 30 seconds
- Code: Saves all results automatically
- **Time: 30 seconds**

Building a Simple Test Runner

```
import openai
import time

openai.api_key = "your-key-here"

def test_prompt(prompt, test_input):
    """Send a prompt and input to the LLM, return response"""
    response = openai.ChatCompletion.create(
        model="gpt-3.5-turbo",
        messages=[
            {"role": "system", "content": prompt},
            {"role": "user", "content": test_input}
        ],
        temperature=0
    )
    return response['choices'][0]['message']['content']

# Test cases
test_cases = [
```

```
    "What's the capital of France?",
    "What's the capital of Japan?",
    "What's the capital of Brazil?"
]

# Prompt to test
prompt = "You are a geography expert. Answer with just the city name, nothing else."

# Run tests
print("Testing prompt:", prompt)
print("-" * 50)

for i, test in enumerate(test_cases):
    print(f"\nTest {i+1}: {test}")
    result = test_prompt(prompt, test)
    print(f"Response: {result}")

    time.sleep(1)  # Pause to respect rate limits
```

Output:

```
Testing prompt: You are a geography expert. Answer with just the city name, nothing else
--------------------------------------------------

Test 1: What's the capital of France?
Response: Paris

Test 2: What's the capital of Japan?
Response: Tokyo

Test 3: What's the capital of Brazil?
Response: Brasília
```

Comparing Multiple Prompt Versions

```
# Multiple versions to compare
prompts = {
    "v1": "Answer this question:",
    "v2": "You are a geography expert. Answer with just the city name.",
    "v3": "Answer with only the city name, no punctuation or explanation."
}

test_question = "What's the capital of France?"
```

```
print(f"Question: {test_question}\n")

for version, prompt in prompts.items():
    response = test_prompt(prompt, test_question)
    print(f"{version}: {response}")
```

Output shows which version gives cleanest results:

```
Question: What's the capital of France?

v1: The capital of France is Paris.
v2: Paris
v3: Paris
```

Hands-On Activity: Build Your Own Test Suite

Task: Test prompt variations for email sentiment analysis

```
# Your test cases
test_emails = [
    "Thank you so much! This solved my problem perfectly!",
    "This is unacceptable. I want a refund immediately.",
    "I received my order. It's fine.",
    "The product is okay but shipping took forever 😞"
]

# Your prompt versions
prompts = [
    "Classify sentiment as positive, negative, or neutral:",

    """Classify email sentiment as:
    - Positive: Customer is happy/satisfied
    - Negative: Customer is unhappy/frustrated
    - Neutral: Factual statement without emotion

    Respond with only one word.""",

    """You are a customer service sentiment analyzer.
    Classify this email as positive, negative, or neutral.
    Consider both explicit words and emotional tone.
    Output only: positive, negative, or neutral"""
]
```

```
# Test all combinations
for i, prompt in enumerate(prompts):
    print(f"\n{'='*50}")
    print(f"PROMPT VERSION {i+1}")
    print('='*50)

    for email in test_emails:
        result = test_prompt(prompt, email)
        print(f"\nEmail: {email[:50]}...")
        print(f"Classification: {result}")

    time.sleep(2)  # Be nice to the API
```

Analyze results:
- Which version was most accurate?
- Which had most consistent formatting?
- Were there any surprises?

Day 18: Saving and Processing Results

Writing Results to Files

Why save results?
- Compare prompt versions over time
- Share results with teammates
- Build datasets for further analysis
- Track costs and performance

Saving to Text Files

```
import openai
from datetime import datetime

openai.api_key = "your-key-here"

# Test a prompt
prompt = "Summarize in one sentence:"
text = "Long article text here..."

response = openai.ChatCompletion.create(
    model="gpt-3.5-turbo",
```

```
    messages=[
        {"role": "system", "content": prompt},
        {"role": "user", "content": text}
    ]
)

result = response['choices'][0]['message']['content']
tokens_used = response['usage']['total_tokens']

# Save to file
timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
with open("test_results.txt", "a") as file:
    file.write(f"\n{'='*50}\n")
    file.write(f"Timestamp: {timestamp}\n")
    file.write(f"Prompt: {prompt}\n")
    file.write(f"Input length: {len(text)} characters\n")
    file.write(f"Tokens used: {tokens_used}\n")
    file.write(f"Result: {result}\n")

print("Results saved to test_results.txt")
```

Saving to CSV (Structured Data)

```
import csv

# Your test results
results = [
    {"prompt_version": "v1", "test_case": "Case 1", "output": "Result 1", "tokens": 45},
    {"prompt_version": "v1", "test_case": "Case 2", "output": "Result 2", "tokens": 52},
    {"prompt_version": "v2", "test_case": "Case 1", "output": "Result 3", "tokens": 38},
]

# Write to CSV
with open("prompt_tests.csv", "w", newline='') as file:
    writer = csv.DictWriter(file, fieldnames=["prompt_version", "test_case", "output", "
    writer.writeheader()
    writer.writerows(results)

print("Results saved to prompt_tests.csv")
```

You can then open this in Excel or Google Sheets!

Tracking Costs

```
# Pricing (per 1,000 tokens)
PRICING = {
    "gpt-3.5-turbo": 0.002,
    "gpt-4": 0.03
}

def calculate_cost(tokens, model):
    """Calculate cost for token usage"""
    price_per_1k = PRICING[model]
    cost = (tokens / 1000) * price_per_1k
    return cost

# Track costs across multiple calls
total_tokens = 0
total_cost = 0

test_cases = ["test 1", "test 2", "test 3"]

for test in test_cases:
    response = openai.ChatCompletion.create(
        model="gpt-3.5-turbo",
        messages=[{"role": "user", "content": test}]
    )

    tokens = response['usage']['total_tokens']
    cost = calculate_cost(tokens, "gpt-3.5-turbo")

    total_tokens += tokens
    total_cost += cost

    print(f"Test: {test} | Tokens: {tokens} | Cost: ${cost:.4f}")

print(f"\nTotal tokens: {total_tokens}")
print(f"Total cost: ${total_cost:.4f}")
```

Hands-On Activity: Build a Cost Tracker

Task: Create a script that tracks cumulative API costs

```
import json
import os
from datetime import datetime
```

```
def load_cost_history():
    """Load previous cost tracking data"""
    if os.path.exists("cost_tracker.json"):
        with open("cost_tracker.json", "r") as f:
            return json.load(f)
    return {"total_spent": 0, "calls": []}

def save_cost_history(data):
    """Save cost tracking data"""
    with open("cost_tracker.json", "w") as f:
        json.dump(data, f, indent=2)

def track_api_call(tokens, model, purpose):
    """Record an API call and its cost"""
    history = load_cost_history()

    cost = calculate_cost(tokens, model)

    call_data = {
        "timestamp": datetime.now().isoformat(),
        "model": model,
        "tokens": tokens,
        "cost": cost,
        "purpose": purpose
    }

    history["calls"].append(call_data)
    history["total_spent"] += cost

    save_cost_history(history)

    print(f"Logged: {purpose} - ${cost:.4f}")
    print(f"Total spent: ${history['total_spent']:.4f}")

# Example usage
track_api_call(450, "gpt-3.5-turbo", "Testing prompt v1")
track_api_call(520, "gpt-3.5-turbo", "Testing prompt v2")
```

Deliverable: Run this for all your Week 3 experiments and review your cost_tracker.json file.

Day 19-21: Week 3 Project

Project: Automated Prompt Comparison System

Goal: Build a Python script that systematically compares prompt versions

Requirements:

1. **Multiple prompt versions** (at least 3)
2. **Test dataset** (at least 10 test cases)
3. **Automated testing** (runs all combinations)
4. **Results saved** (CSV or JSON format)
5. **Cost tracking** (total tokens and $ spent)
6. **Summary report** (which prompt performed best)

Starter Code:

```
import openai
import csv
import time
from datetime import datetime

openai.api_key = "your-key-here"

# Configuration
MODEL = "gpt-3.5-turbo"
TEMPERATURE = 0

# Prompt versions to compare
PROMPTS = {
    "v1_basic": "Classify sentiment as positive, negative, or neutral:",

    "v2_detailed": """Classify customer email sentiment:
    - positive: Customer is satisfied
    - negative: Customer is unsatisfied
    - neutral: Neither positive nor negative

    Respond with only the classification.""",

    "v3_fewshot": """Classify sentiment as positive, negative, or neutral.

    Examples:
    Email: "Love it!" → positive
    Email: "Terrible service" → negative
```

```
    Email: "Order received" → neutral

    Now classify this email:"""
}

# Test cases
TEST_CASES = [
    {"email": "Thank you! Exactly what I needed!", "expected": "positive"},
    {"email": "Completely disappointed. Refund please.", "expected": "negative"},
    {"email": "Package arrived on Tuesday.", "expected": "neutral"},
    # Add 7 more test cases here
]

def test_prompt(prompt, test_input):
    """Run a single test"""
    try:
        response = openai.ChatCompletion.create(
            model=MODEL,
            messages=[
                {"role": "system", "content": prompt},
                {"role": "user", "content": test_input}
            ],
            temperature=TEMPERATURE
        )

        result = response['choices'][0]['message']['content'].strip().lower()
        tokens = response['usage']['total_tokens']

        return result, tokens
    except Exception as e:
        return f"ERROR: {str(e)}", 0

def run_full_comparison():
    """Run all prompt versions against all test cases"""
    results = []

    print("Starting comparison...")
    print(f"Testing {len(PROMPTS)} prompts on {len(TEST_CASES)} cases\n")

    for prompt_name, prompt_text in PROMPTS.items():
        print(f"\nTesting: {prompt_name}")
        print("-" * 50)

        correct = 0
        total_tokens = 0
```

```
        for i, test in enumerate(TEST_CASES):
            output, tokens = test_prompt(prompt_text, test["email"])

            is_correct = test["expected"] in output
            if is_correct:
                correct += 1

            total_tokens += tokens

            results.append({
                "prompt_version": prompt_name,
                "test_number": i + 1,
                "input": test["email"][:50],
                "expected": test["expected"],
                "output": output,
                "correct": is_correct,
                "tokens": tokens
            })

            print(f"  Test {i+1}: {'✓' if is_correct else '✗'} ({output})")
            time.sleep(1)  # Rate limiting

        accuracy = (correct / len(TEST_CASES)) * 100
        cost = (total_tokens / 1000) * 0.002  # GPT-3.5 pricing

        print(f"\n{prompt_name} Summary:")
        print(f"  Accuracy: {accuracy:.1f}%")
        print(f"  Total tokens: {total_tokens}")
        print(f"  Cost: ${cost:.4f}")

    return results

def save_results(results):
    """Save results to CSV"""
    filename = f"prompt_comparison_{datetime.now().strftime('%Y%m%d_%H%M%S')}.csv"

    with open(filename, 'w', newline='') as f:
        writer = csv.DictWriter(f, fieldnames=results[0].keys())
        writer.writeheader()
        writer.writerows(results)

    print(f"\nResults saved to: {filename}")

# Run the comparison
if __name__ == "__main__":
    results = run_full_comparison()
```

```
    save_results(results)

    print("\n" + "="*50)
    print("Comparison complete!")
    print("="*50)
```

Your Tasks:

1. **Customize the prompts** to something relevant to you (email classification, text summarization, etc.)
2. **Add at least 10 test cases** with expected outputs
3. **Run the comparison** and save results
4. **Analyze results:**
   - Which prompt had highest accuracy?
   - Which used fewest tokens?
   - What was the cost difference?
   - Which would you choose for production?
5. **Document findings** in a simple report:
   ```
   PROMPT COMPARISON REPORT
   Date: [date]
   Task: [what you tested]

RESULTS:
v1_basic: 60% accuracy, 2,340 tokens, $0.0047
v2_detailed: 85% accuracy, 3,120 tokens, $0.0062
v3_fewshot: 90% accuracy, 4,560 tokens, $0.0091

RECOMMENDATION:
Use v2_detailed because... [your reasoning]
```

Week 4: Putting It All Together

Learning Objectives

By the end of this week, you will:
- Combine manual prompting skills with programming automation

- Understand when to use different approaches
- Build a complete prompt engineering workflow
- Prepare for advanced topics (RAG, fine-tuning, production systems)

Day 22: Building Complete Workflows

What is a Workflow?

Simple task: Single prompt → Single response

Workflow: Multiple steps with logic

Example workflow: Article Processing

```
1. Input: Raw article URL
2. Step 1: Extract text from URL
3. Step 2: Summarize article
4. Step 3: Extract key entities (people, companies)
5. Step 4: Generate social media posts
6. Step 5: Create image caption ideas
7. Output: Comprehensive package
```

Sequential Workflows

Pattern: Output of step N becomes input to step N+1

```
def process_article(url):
    # Step 1: Get article text (assume function exists)
    text = extract_text_from_url(url)

    # Step 2: Summarize
    summary = llm_call(
        prompt="Summarize in 3 sentences:",
        input=text
    )

    # Step 3: Extract entities from summary
    entities = llm_call(
        prompt="Extract key people and organizations mentioned:",
        input=summary
    )
```

```
    # Step 4: Generate tweet using summary and entities
    tweet = llm_call(
        prompt=f"""Create an engaging tweet about this article.
        Summary: {summary}
        Key entities: {entities}
        """,
        input=""
    )

    return {
        "summary": summary,
        "entities": entities,
        "tweet": tweet
    }
```

Benefits:

- Each step is simple and testable
- Failures are isolated and debuggable
- Can optimize each step independently

Parallel Workflows

Pattern: Run multiple independent tasks simultaneously

```
def analyze_product_review(review):
    # These can run in parallel (don't depend on each other)

    # Task 1: Sentiment
    sentiment = llm_call("Classify sentiment:", review)

    # Task 2: Extract product mentioned
    product = llm_call("What product is being reviewed?", review)

    # Task 3: Extract pros/cons
    pros_cons = llm_call("List pros and cons mentioned:", review)

    # Task 4: Rating prediction
    predicted_rating = llm_call("Predict star rating (1-5):", review)

    return {
        "sentiment": sentiment,
        "product": product,
        "pros_cons": pros_cons,
```

```
        "rating": predicted_rating
    }
```

Conditional Workflows

Pattern: Different paths based on outputs

```
def route_customer_email(email):
    # Step 1: Classify urgency
    urgency = llm_call("Classify urgency as high, medium, or low:", email)

    if "high" in urgency.lower():
        # High urgency: Escalate and draft urgent response
        department = "priority_support"
        response = llm_call(
            "Draft an immediate response acknowledging urgency:",
            email
        )
    elif "medium" in urgency.lower():
        # Medium: Standard routing
        department = llm_call(
            "Which department: billing, technical, or sales?",
            email
        )
        response = llm_call(
            "Draft a professional response:",
            email
        )
    else:
        # Low: Auto-response
        department = "general"
        response = "Thank you for contacting us. We'll respond within 24 hours."

    return {
        "urgency": urgency,
        "department": department,
        "draft_response": response
    }
```

Hands-On Activity: Design Your Workflow

Task: Map out a workflow for a real task from your life

Template:

```
WORKFLOW NAME: [descriptive name]

INPUT: [what starts the workflow]

STEPS:
1. [Step name]: [What happens] → [Output]
2. [Step name]: [What happens] → [Output]
3. [Conditional/Parallel?]: [Branch logic if applicable]
...

FINAL OUTPUT: [What the workflow produces]

SUCCESS CRITERIA: [How do you know it worked?]
```

Example domains:
- Content creation pipeline
- Research synthesis workflow
- Customer support routing
- Document processing
- Data extraction and formatting

Deliverable: Write out your workflow design (no code yet, just the plan)

Day 23: Error Handling and Edge Cases

Why Things Fail

Common failure modes:
1. **API errors** (rate limits, network issues, invalid keys)
2. **Unexpected outputs** (wrong format, hallucinations)
3. **Edge cases** (empty inputs, very long inputs)
4. **Cost overruns** (accidentally expensive calls)

Defensive Programming

Pattern 1: Try-Except Blocks

```
def safe_llm_call(prompt, input_text):
    """LLM call with error handling"""
    try:
        response = openai.ChatCompletion.create(
```

```
            model="gpt-3.5-turbo",
            messages=[
                {"role": "system", "content": prompt},
                {"role": "user", "content": input_text}
            ]
        )
        return response['choices'][0]['message']['content']

    except openai.error.RateLimitError:
        print("Rate limit hit. Waiting 60 seconds...")
        time.sleep(60)
        return safe_llm_call(prompt, input_text)  # Retry

    except openai.error.InvalidRequestError as e:
        print(f"Invalid request: {e}")
        return "ERROR: Invalid request"

    except Exception as e:
        print(f"Unexpected error: {e}")
        return "ERROR: Call failed"
```

Pattern 2: Input Validation

```
def validate_input(text, max_length=10000):
    """Check input before sending to API"""

    # Check if empty
    if not text or len(text.strip()) == 0:
        return False, "Input is empty"

    # Check length
    if len(text) > max_length:
        return False, f"Input too long ({len(text)} chars, max {max_length})"

    # Check for suspicious content
    suspicious_keywords = ["ignore previous", "system:", "jailbreak"]
    if any(keyword in text.lower() for keyword in suspicious_keywords):
        return False, "Input contains suspicious content"

    return True, "Valid"

# Usage
user_input = "Some user text..."
is_valid, message = validate_input(user_input)
```

```
if is_valid:
    result = safe_llm_call(prompt, user_input)
else:
    print(f"Invalid input: {message}")
```

Pattern 3: Output Validation

```
def validate_sentiment_output(output):
    """Ensure sentiment classification is valid"""
    valid_sentiments = ["positive", "negative", "neutral"]

    output_clean = output.strip().lower()

    if output_clean in valid_sentiments:
        return True, output_clean
    else:
        # Try to find valid sentiment in response
        for sentiment in valid_sentiments:
            if sentiment in output_clean:
                return True, sentiment

        # Couldn't validate
        return False, None

# Usage
llm_output = "The sentiment is positive!"
is_valid, sentiment = validate_sentiment_output(llm_output)

if is_valid:
    print(f"Classified as: {sentiment}")
else:
    print("Invalid output, retrying with stricter prompt...")
```

Pattern 4: Retry with Backoff

```
import time

def llm_call_with_retry(prompt, input_text, max_retries=3):
    """Retry failed calls with exponential backoff"""

    for attempt in range(max_retries):
        try:
            response = openai.ChatCompletion.create(
                model="gpt-3.5-turbo",
```

```
                messages=[
                    {"role": "system", "content": prompt},
                    {"role": "user", "content": input_text}
                ]
            )
            return response['choices'][0]['message']['content']

        except Exception as e:
            if attempt < max_retries - 1:
                wait_time = 2 ** attempt  # 1s, 2s, 4s, 8s...
                print(f"Attempt {attempt + 1} failed. Waiting {wait_time}s...")
                time.sleep(wait_time)
            else:
                print(f"All {max_retries} attempts failed.")
                return None
```

Hands-On Activity: Break Your Code (Then Fix It)

Task: Intentionally test failure modes

```
# Test 1: Empty input
result = safe_llm_call("Summarize:", "")
# What happens? How should you handle it?

# Test 2: Extremely long input
very_long_text = "word " * 100000  # Way too long
result = safe_llm_call("Summarize:", very_long_text)
# What happens? How do you prevent this?

# Test 3: Invalid API key
old_key = openai.api_key
openai.api_key = "invalid-key"
result = safe_llm_call("Test", "test")
openai.api_key = old_key
# Does your error handling catch this?

# Test 4: Unexpected output format
result = safe_llm_call("Respond with only 'yes' or 'no':", "Maybe?")
# Does your validation catch non-compliant outputs?
```

Document:

- What errors occurred?
- How did your code handle them?
- What additional handling did you add?

Day 24: When to Use What Approach

Decision Framework

Choose manual prompting (ChatGPT interface) when:
- ✅ One-off tasks
- ✅ Exploring/brainstorming
- ✅ Learning what works
- ✅ Need quick answer
- ✅ Task has high variation

Choose programmatic approach (API) when:
- ✅ Repetitive tasks (>10 similar items)
- ✅ Need consistency
- ✅ Processing at scale
- ✅ Integration with other systems
- ✅ Need to track metrics/costs

Choose workflows (multi-step) when:
- ✅ Task has clear stages
- ✅ Output of one step feeds another
- ✅ Need error handling at each stage
- ✅ Different steps need different models/settings

Real-World Scenarios

Scenario 1: Write one marketing email
→ **Use:** ChatGPT interface
→ **Why:** One-off, creative task, lots of back-and-forth

Scenario 2: Classify 500 customer reviews
→ **Use:** Python script with API
→ **Why:** Repetitive, needs consistency, scalable

Scenario 3: Daily digest of news articles
→ **Use:** Automated workflow
→ **Why:** Repetitive, multi-step (fetch → summarize → format), scheduled

Scenario 4: Draft responses to support tickets
→ **Use:** Hybrid (API generates draft, human reviews)
→ **Why:** Consistency + human judgment needed

Scenario 5: Experiment with prompt variations
→ **Use:** Python test script from Week 3
→ **Why:** Need systematic comparison with metrics

Cost Considerations

Manual approach costs:
- Your time (high for repetitive tasks)
- No token costs (with free tier)
- Difficult to measure ROI

Programmatic approach costs:
- API tokens (measurable, scalable)
- Development time (upfront investment)
- Easy to measure ROI

Example calculation:

```
Task: Classify 100 emails daily

Manual approach:
- 2 minutes per email = 200 minutes = 3.3 hours
- Cost: 3.3 hours × $50/hour = $165/day
- Monthly: $3,300

Programmatic approach:
- Development: 4 hours × $50/hour = $200 (one-time)
- API costs: 100 calls × 400 tokens × $0.002/1k = $0.08/day
- Monthly: $0.08 × 20 days = $1.60

ROI: Pays for itself in < 2 days
```

Hands-On Activity: Audit Your Tasks

Task: Identify tasks that should be automated

Template:

```
TASK AUDIT

Task: [What you do]
Frequency: [How often]
Current approach: [Manual/Code/Mixed]
Time per instance: [Minutes]
```

```
Monthly volume: [How many times]

Should automate? [Yes/No/Maybe]
Reasoning: [Why or why not]
Potential savings: [Time/money]
```

Example:

```
Task: Summarize weekly industry news
Frequency: Weekly (4x/month)
Current: Manually read and write summaries
Time: 2 hours per week
Monthly volume: 8 hours

Should automate? YES
Reasoning: Consistent format, high time cost, easy to structure
Potential savings: 6-7 hours/month (automate reading/drafting, keep human review)
```

Your tasks to audit: List 5-10 tasks you do regularly

Day 25: Advanced Concepts Preview

What You're Ready For Next

After completing this foundations course, you can tackle:

1. Production Prompt Engineering (Subject 1 - the document you're preparing for)
- Systematic testing with metrics
- Version control and deployment
- Cost optimization
- Failure mode analysis

2. Retrieval-Augmented Generation (RAG)
- Give LLMs access to your specific documents
- Build Q&A systems over your data
- Combine search with generation

3. Fine-Tuning
- Customize models for specific tasks
- Improve quality beyond prompting
- Reduce costs for high-volume tasks

4. LLM Application Development
- Build user-facing tools
- Create APIs and integrations
- Deploy production systems

Concepts You've Learned (Celebrate!)

Week 1: Understanding
- ✅ What LLMs are and how they work
- ✅ Limitations and capabilities
- ✅ Safety and ethics considerations

Week 2: Prompting Skills
- ✅ Writing effective prompts
- ✅ Few-shot learning
- ✅ Chain-of-thought reasoning
- ✅ Systematic iteration

Week 3: Programming
- ✅ Python basics
- ✅ API integration
- ✅ Automated testing
- ✅ Cost tracking

Week 4: Production Thinking
- ✅ Workflow design
- ✅ Error handling
- ✅ When to use different approaches

Hands-On Activity: Self-Assessment

Rate your understanding (1-5 scale):

```
CORE CONCEPTS
__ What LLMs are and their limitations
__ When AI is/isn't appropriate for a task
__ Ethical considerations in AI use

PROMPTING SKILLS
__ Writing clear, specific prompts
__ Using examples effectively
__ Iterating based on results
__ Chain-of-thought reasoning
```

```
TECHNICAL SKILLS
__ Basic Python programming
__ Making API calls
__ Automating repetitive tasks
__ Handling errors

PRODUCTION THINKING
__ Designing workflows
__ Validating inputs/outputs
__ Cost-benefit analysis
__ When to automate vs. manual

NEXT STEPS
__ Ready for advanced prompt engineering (Subject 1)
__ Need more practice with programming
__ Want to explore specific applications
__ Ready to build real projects
```

If you rated most items 3+: You're ready for Subject 1!

If you have multiple 1-2 ratings: Review those specific days/concepts before moving on.

Day 26-28: Final Project

Capstone: Build a Complete Application

Goal: Create an end-to-end prompt-powered tool that demonstrates everything you've learned.

Requirements:

1. **Solves a real problem** (from your life/work)
2. **Uses programmatic prompting** (Python + API)
3. **Multi-step workflow** (at least 3 steps)
4. **Error handling** (validates inputs and outputs)
5. **Saves results** (to file or displays clearly)
6. **Documented** (README explaining what it does and how to use it)

Project Ideas:

Idea 1: Article Research Assistant

```
Input: Topic or URL
Steps:
1. Extract main article content
2. Generate 3-sentence summary
3. Extract key people, companies, concepts
4. Generate 5 follow-up questions
5. Create social media post
Output: Formatted report with all elements
```

Idea 2: Email Response Generator

```
Input: Customer email
Steps:
1. Classify urgency and category
2. Extract key customer concerns
3. Generate appropriate response
4. Suggest follow-up actions
Output: Categorization + draft response
```

Idea 3: Content Repurposing Tool

```
Input: Long-form content (blog post)
Steps:
1. Generate summary (3 lengths: short/medium/long)
2. Extract quotable insights
3. Create Twitter thread (5-7 tweets)
4. Generate LinkedIn post
5. Suggest hashtags
Output: Multi-platform content package
```

Idea 4: Learning Flashcard Generator

```
Input: Educational article or textbook passage
Steps:
1. Extract key concepts
2. Generate definition for each
3. Create question/answer pairs
4. Suggest mnemonic devices
Output: Study materials in flashcard format
```

Choose your own! What would genuinely help you?

Starter Template

```
"""
[YOUR PROJECT NAME]
[Brief description of what it does]

Author: [Your name]
Date: [Date]
"""

import openai
import json
from datetime import datetime

# Configuration
openai.api_key = "your-key-here"
MODEL = "gpt-3.5-turbo"

def validate_input(input_text):
    """Validate user input before processing"""
    # Add your validation logic
    pass

def step_1(input_data):
    """[Describe what this step does]"""
    prompt = "[Your prompt here]"
    # Call LLM
    # Validate output
    # Return result
    pass

def step_2(previous_output):
    """[Describe what this step does]"""
    # Build on previous step
    pass

def step_3(previous_output):
    """[Describe what this step does]"""
    pass

def run_complete_workflow(user_input):
    """Run all steps in sequence"""
    print("Starting workflow...")

    # Validate
```

```
    if not validate_input(user_input):
        return {"error": "Invalid input"}

    # Execute steps
    try:
        result_1 = step_1(user_input)
        print("✓ Step 1 complete")

        result_2 = step_2(result_1)
        print("✓ Step 2 complete")

        result_3 = step_3(result_2)
        print("✓ Step 3 complete")

        # Compile final output
        final_output = {
            "timestamp": datetime.now().isoformat(),
            "input": user_input,
            "result_1": result_1,
            "result_2": result_2,
            "result_3": result_3
        }

        return final_output

    except Exception as e:
        return {"error": str(e)}

def save_results(results, filename):
    """Save results to file"""
    with open(filename, 'w') as f:
        json.dump(results, f, indent=2)
    print(f"Results saved to {filename}")

# Main execution
if __name__ == "__main__":
    # Get user input
    user_input = input("Enter your input: ")

    # Run workflow
    results = run_complete_workflow(user_input)

    # Display results
    print("\n" + "="*50)
    print("RESULTS")
    print("="*50)
```

```
    print(json.dumps(results, indent=2))

    # Save results
    save_results(results, "output.json")
```

Deliverables

1. Working Python script (commented and organized)

2. README.md document:

```
# [Project Name]

## What It Does
[2-3 sentence description]

## Why I Built It
[What problem does it solve?]

## How It Works
1. [Step 1 description]
2. [Step 2 description]
3. [Step 3 description]

## How to Use
1. Install requirements: `pip install openai`
2. Set API key: [instructions]
3. Run: `python your_script.py`
4. Enter input when prompted

## Example Output
[Show what the output looks like]

## What I Learned
[Reflect on what you learned building this]

## Future Improvements
[What would you add with more time/skills?]
```

3. Example outputs (Run your tool 3-5 times, save the results)

4. Reflection document:

```
PROJECT REFLECTION

What went well:
- [What worked better than expected?]
- [What are you proud of?]

Challenges faced:
- [What was difficult?]
- [How did you overcome it?]

Prompt evolution:
- [How did your prompts change from first to final version?]
- [What improved results most?]

Cost analysis:
- Total tokens used: [number]
- Total cost: $[amount]
- Average cost per run: $[amount]

Ready for Subject 1?
- [Yes/No and why]
- [What topics are you most excited about?]
- [Any remaining gaps in understanding?]
```

Course Complete! 🎉

What You've Achieved

You started with: Curiosity about AI

You now have:
- ✅ Deep understanding of LLMs and their capabilities
- ✅ Systematic prompting skills with proven techniques
- ✅ Programming abilities to automate LLM workflows
- ✅ Production thinking about costs, errors, and validation
- ✅ A portfolio project demonstrating competence
- ✅ Foundation for advanced prompt engineering

Your Next Steps

Immediate (This Week):
1. Complete your final project if not done
2. Review any concepts that still feel unclear
3. Share your project (GitHub, personal site, etc.)

Short-Term (Next Month):
1. Start **Subject 1: Prompt Engineering as an Engineering Discipline**
2. Practice building more complex workflows
3. Explore one advanced topic (RAG, fine-tuning, or agents)

Long-Term (Next 3-6 Months):
1. Build 2-3 production-quality projects
2. Contribute to open-source LLM projects
3. Consider: Job opportunities, freelancing, or building products

Resources for Continued Learning

Communities:
- OpenAI Developer Forum
- r/PromptEngineering
- LangChain Discord

Tutorials & Courses:
- DeepLearning.AI Short Courses (Free)
- Prompt Engineering Guide (Comprehensive)
- OpenAI Cookbook (Practical examples)

Stay Updated:
- Import AI Newsletter
- The Batch (DeepLearning.AI)
- Follow researchers on Twitter/X (Andrej Karpathy, Jeremy Howard, etc.)

Practice Platforms:
- LeetCode for Prompt Engineering
- Kaggle LLM Competitions
- Build tools for your own use cases!

Final Encouragement

You've completed a challenging journey from zero AI knowledge to functional prompt engineering skills. Most importantly, you've learned **how to learn** in this rapidly evolving field.

Remember:
- 🎯 **Start simple, iterate constantly**
- 📊 **Measure everything you can**
- 🔧 **Build real things for real problems**
- 🤝 **Share what you learn with others**
- 🚀 **Stay curious and keep experimenting**

The field of AI is moving fast. Your ability to learn, adapt, and build will serve you far better than any single technique or tool.

You're ready. Go build something amazing.

Appendix: Quick Reference

Common Prompt Patterns

Summarization:

```
Summarize the following in [N] sentences/words:
- Focus on: [key aspects]
- Audience: [who is this for]
- Tone: [formal/casual/etc]

[TEXT]
```

Classification:

```
Classify the following as [CATEGORY_1], [CATEGORY_2], or [CATEGORY_3]:

Criteria:
- [CATEGORY_1]: [definition]
- [CATEGORY_2]: [definition]
- [CATEGORY_3]: [definition]

Respond with only the category name.

[INPUT]
```

Extraction:

```
Extract the following information in this exact format:

[FIELD_1]: [description]
[FIELD_2]: [description]

If information is missing, write "Not found"

[TEXT]
```

Generation:

```
[ROLE]: You are [detailed persona]
[TASK]: Create [specific output]
[CONTEXT]: This will be used for [purpose]
[CONSTRAINTS]:
- [constraint 1]
- [constraint 2]
[FORMAT]: [output structure]

[INPUT/SEED]
```

Python Snippets

Basic API Call:

```
import openai
openai.api_key = "your-key"

response = openai.ChatCompletion.create(
    model="gpt-3.5-turbo",
    messages=[{"role": "user", "content": "Your prompt"}]
)

result = response['choices'][0]['message']['content']
```

With Error Handling:

```
try:
    response = openai.ChatCompletion.create(...)
    result = response['choices'][0]['message']['content']
except Exception as e:
```

```
    print(f"Error: {e}")
    result = None
```

Save to File:

```
with open("output.txt", "w") as f:
    f.write(result)
```

Cost Calculation:

```
tokens = response['usage']['total_tokens']
cost = (tokens / 1000) * 0.002  # GPT-3.5 pricing
print(f"Cost: ${cost:.4f}")
```

Troubleshooting Guide

Problem: API returns error 401
→ Check API key is correct and has credits

Problem: Outputs are inconsistent
→ Lower temperature (try 0.0)
→ Add more specific constraints
→ Use few-shot examples

Problem: Outputs wrong format
→ Show explicit example format
→ Add validation and retry logic
→ Use structured output mode if available

Problem: Too expensive
→ Use GPT-3.5 instead of GPT-4
→ Shorten prompts (remove unnecessary context)
→ Cache repeated prompt segments
→ Pre-filter inputs (don't process everything)

Problem: Too slow
→ Use faster model (GPT-3.5-turbo)
→ Reduce max_tokens
→ Process items in parallel (advanced)

Problem: Hallucinations/wrong info
→ Add "If you don't know, say so"

→ Lower temperature
→ Request citations/sources
→ Add verification step

Version: 1.0
Last Updated: 2024
Next Document: Subject 1 - Prompt Engineering as an Engineering Discipline
Contact: [How to reach you for questions]

This document is designed to prepare learners for production-level prompt engineering. It covers foundational concepts systematically while building practical skills through hands-on projects. Upon completion, learners should be ready to tackle advanced topics including systematic testing, version control, cost optimization, and production deployment.